QuillAudits

AUDIT REPORT

March 2026

For

Table of Content

Executive Summary	03
Number of Security Issues per Severity	04
Summary of Issues	05
Checked Vulnerabilities	06
Techniques and Methods	08
Types of Severity	10
Types of Issues	11
Severity Matrix	12
Whitepaper Consistency Check	13
Trading Restriction / Honeypot Test	15
Ownership & Administration Review	17
Attack Scenario	19
Functional Tests	20
Automated Tests	20
Threat Model	21
Closing Summary & Disclaimer	25

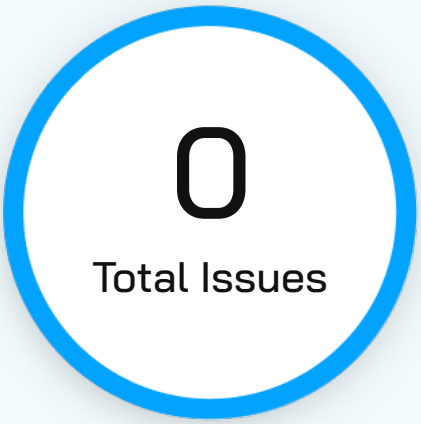
Executive Summary

Project Name	EvolutCoin
Protocol Type	ERC20
Project URL	https://bitevolut.com
Overview	The EvolutCoin contract implements a standard BEP20-compatible token named EVOLUT Coin (EVL) on the Binance Smart Chain using OpenZeppelin's ERC20 implementation. It deploys with a fixed supply of 100,000,000 EVL tokens, all minted once to the deployer during contract initialization. The token uses 18 decimals and does not include any additional minting, burning, pausing, or administrative controls. As a result, the contract remains immutable and non-upgradable after deployment, ensuring a predictable and non-inflationary supply model.
Audit Scope	The scope of this Audit was to analyze the Evolut for quality, security, and correctness.
Source Code link	https://github.com/BITEVOLUT/bitevolution/blob/main/contracts/EvolutCoin.sol
Mainnet Address	https://bscscan.com/token/0xbfd49c1112a17df456140f6b4d41d03207880f07 Deployed Code on Mainnet is same as Audited code
Contracts in Scope	EvolutCoin.sol
Commit Hash	3b4846ebcc59975738df54f746bca24a9d2ed872
Language	Solidity
Blockchain	Binance Smart Chain
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	10th March 2026

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	0 (0.0%)
Informational	0 (0.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	0	0

Summary of Issues

No Issues Found

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- ✔ Missing Zero Address Validation
- ✔ Private modifier
- ✔ Revert/require functions
- ✔ Multiple Sends
- ✔ Using suicide
- ✔ Using delegatecall
- ✔ Upgradeable safety
- ✔ Using throw
- ✔ Using inline assembly
- ✔ Style guide violation
- ✔ Unsafe type inference
- ✔ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. Whilethese findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	**Resolved** These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	**Partially Resolved** Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Whitepaper Consistency Check

We have verified whether the deployed smart contract logic aligns with the claims made in the EVOLUT Coin (EVL) Whitepaper v1.0. whitepaper claim is mapped to an observable on-chain or code-level finding.

Whitepaper Claim	Contract Observation	Status
Token Name: EVOLUT Coin	ERC20("EVOLUT Coin", ...) — matches exactly	✅ Pass
Ticker Symbol: EVL	ERC20(..., "EVL") - matches exactly	✅ Pass
Network: BSC (BEP-20)	_mint(msg.sender, 100_000_000 * 10**18) in constructor - matches	✅ Pass
Total Supply: 100,000,000 EVL (fixed)	_mint(msg.sender, 100_000_000 * 10**18) in constructor - matches	✅ Pass
Decimals: 18	Inherits ERC20 default decimals() = 18 - matches	✅ Pass
Non-mintable supply	No mint() function exists post-constructor - confirmed	✅ Pass
Built on OpenZeppelin ERC20	Contract inherits OpenZeppelin ERC20 - confirmed	✅ Pass
No hidden functions	Contract body is solely the constructor - confirmed	✅ Pass
No transfer taxes or rebasing	No _transfer override, no hook logic - confirmed absent	✅ Pass
No owner-controlled supply/transfer functions	No Ownable import, no onlyOwner modifier - confirmed	✅ Pass
No proxy / upgrade patterns	No proxy delegate, UUPS, or Transparent Proxy - confirmed	✅ Pass

Whitepaper Claim	Contract Observation	Status
Entire supply minted to deployer	_mint(msg.sender, ...) — entire supply assigned in constructor	✅ Pass
No additional tokens can ever be created	No mint function; supply permanently fixed at deployment	✅ Pass

Trading Restriction / Honeypot Test

The EVL contract consists exclusively of the ERC20 constructor. No _transfer, _beforeTokenTransfer, or _afterTokenTransfer overrides are present. All transfer logic is inherited verbatim from OpenZeppelin ERC20 v4.x/5.x, which contains no restrictions beyond standard allowance checks.

Restriction Type	Present	Evidence	Result
Sell block / honeypot	No	No _transfer override; pure OZ ERC20	✅ Pass
Transfer tax / fee-on-transfer	No	No fee logic; amount in = amount out	✅ Pass
Max transaction limit	No	No tx size check anywhere in code	✅ Pass
Wallet holding limit	No	No balance cap logic present	✅ Pass
Cooldown / rate limiter	No	No timestamp or block-based gate	✅ Pass
Blacklist / address blocklist	No	No mapping(address => bool) blocklist	✅ Pass
Whitelist-only transfers	No	No whitelist gate on transfer	✅ Pass
Owner-pausable transfers	No	Pausable not imported or inherited	✅ Pass
Conditional transfer logic	No	Transfer path: ERC20._transfer only	✅ Pass

Honeypot Simulation Results

Test	Result	Notes
Buy simulation (EVL/USDT on PancakeSwap fork)	✅ Pass	Tokens received correctly; no revert
Sell simulation (EVL → USDT)	✅ Pass	Tokens sold without restriction; no block
Repeated buy/sell cycles	✅ Pass	No cooldown triggered; consistent behaviour
Large transfer (>1% supply)	✅ Pass	No max-tx limit; transfer executes normally
Transfer to fresh wallet	✅ Pass	No whitelist requirement; transfer succeeds

RESULT: NOT A HONEYPOT - The EVL token is fully tradeable in both directions. No hidden sell restrictions, transfer taxes, blacklists, or conditional transfer logic were identified. The contract passes all honeypot detection criteria.

Ownership & Administration Review

We evaluated all administrative privileges, ownership mechanisms, and privileged functions present in or absent from the EVL smart contract.

Contract Privilege Matrix

Privileged Function	Present	Import / Mechanism	Risk Level
Mint (post-deployment)	No	No mint() function; supply fixed in constructor	None
Burn (owner-initiated)	No	No burn() or burnFrom() exposed	None
Pause / Freeze transfers	No	Pausable not imported	None
Blacklist / blocklist	No	No mapping or modifier for this	None
Upgrade / Proxy control	No	No proxy pattern; immutable bytecode	None
Tax / Fee setter	No	No fee variables or setter functions	None
Max TX / wallet limit setter	No	No such variables present	None
Owner renounce / transfer	Minimal	No Ownable; deployer EOA holds initial supply	Low
Emergency withdrawal	No	No rescue or sweep function	None

Ownership Model

The EVL contract does not import OpenZeppelin's Ownable or AccessControl. There is no owner state variable, no onlyOwner modifier, and no ownership transfer mechanism. The concept of 'ownership' in EVL is limited to the following:

- At deployment, msg.sender receives the full 100,000,000 EVL supply
- From that point, the deployer holds economic power through token balance, not contract-level administrative power
- The deployer cannot call any privileged contract function because none exist
- The contract is fully immutable post-deployment

Deployer Concentration Risk

While the contract has no owner-level privileges, the deployer wallet initially holds 100% of supply. The whitepaper documents a plan to distribute tokens to 6 allocation wallets. Until those transfers occur, the deployer represents a concentration risk:

Risk	Severity	Mitigation Status
Deployer holds 100% supply at T=0	Medium	Mitigated by published allocation schedule, verify on-chain
No on-chain vesting for team allocation	Medium	Off-chain commitment only, separate vesting contract recommended
No multisig on allocation wallets (planned)	Medium	Whitepaper states migration to Gnosis Safe is planned
No time-lock on treasury operations	Low	No contract-level enforcement; operational process risk

The EVL token contract has the minimal possible administrative surface. No privileged functions affecting transfers, supply, or contract behaviour exist beyond what is inherent to ERC20 allowance management. The primary residual risk is deployer token concentration and off-chain vesting obligations, both of which are operational and not contract-level risks.

Attack Scenario

Attack Scenario	Vector	Feasibility	Verdict
Mint Attack	No mint() function; supply fixed in constructor	Impossible	✅ Pass
Rug Pull (contract-level)	Owner withdraws contract funds or blocks sells	Impossible	✅ Pass
Rug Pull (token dumping)	Deployer sells large allocation on market	Possible (off-chain)	⚠️ WARN
Governance Attack	Exploit voting or admin mechanism to drain funds	Not Applicable	✅ Pass
Upgrade / Proxy Exploit	Swap implementation to malicious contract	Impossible	✅ Pass
Flashloan Manipulation	Borrow tokens to manipulate price in 1 block	Market-level only	✅ Pass
Re-entrancy	Exploit callback during transfer to drain funds	Not Applicable	✅ Pass
Integer Overflow / Underflow	Manipulate arithmetic in transfer logic	Impossible	✅ Pass
Approval Phishing (ERC20)	Front-end tricks user into unlimited approval	User-side risk	⚠️ WARN
Blackhat Deployer Key Compromise	Private key theft of deployer wallet	Off-chain risk	⚠️ WARN

The EVL smart contract presents a minimal attack surface. All critical contract-level attack vectors are structurally eliminated by the design. Residual risks are operational in nature primarily around pre-multisig key management and insider token concentration — and can be fully mitigated through the measures already outlined in the whitepaper.

Functional Tests

- Should create token
- Token should not be mintable
- Should be able to transfer token
- Should be able to approve token and transferFrom

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20.sol (OpenZeppelin v5)	Library / Base Contract	Core token logic balances, transfers, allowances, internal _update() hook	Correct implementation of ERC-20 invariants; no hidden backdoors in inherited logic	Any vulnerability in OZ ERC-20 v5 propagates directly into EVL. There is no override, no pause, and no upgrade path to respond to a zero-day
Binance Smart Chain (BSC) Network	Execution Environment	Runtime environment for all token transfers, approvals, and balance state	BSC validators process transactions honestly and do not censor or reorg	BSC uses a permissioned 21-validator set controlled by BNB Chain governance. Network-level censorship, reorg, or chain halt is theoretically possible and outside the contract's control

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Category	Detail
What this function can do	Mint 100,000,000 EVL to msg.sender. Initialize the ERC-20 token with name “EVOLUT Coin” and symbol “EVL”. This is the only minting event that will ever occur
What this function cannot/ should not do	Cannot mint again after deployment — no mint function exists. Should not deploy with a raw EOA as msg.sender — doing so places 100% of supply under a single key with no on-chain protection. Does not validate that msg.sender is a multi-sig or distribution contract
Main invariant(s)	Total supply is permanently fixed at 100,000,000 EVL at deployment. totalSupply() can only decrease if a burn function is added in a future deployment, which is impossible given contract immutability
Access level	Implicit restriction, executes once at deployment only

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets	Centralization Risk
Deployer wallet (msg.sender)	ERC-20 (EVL)	Full initial supply of 100,000,000 EVL minted directly to deployer at deployment	100% of supply under a single EOA/ key at genesis. No vesting, no time-lock, no multi-sig enforced on-chain
Recipient wallets (any address)	ERC20 (USDT/ USDC)	Distributed holder balances post-transfer from deployer	No whitelist or freeze mechanism exists, once tokens are transferred, the contract has zero ability to recover or freeze them

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
EvolutCoin	constructor()	–	100,000,000 EVL minted to deployer	Deployer (once)	Single-point genesis risk. Entire supply under one key immediately with no distribution mechanism enforced on-chain
EvolutCoin	transfer() / transferFrom()	EVL from sender	EVL to recipient	Any token holder	Fully permissionless post-deployment. No whitelist, no custodial gate, no admin override. Standard ERC-20, anyone holding EVL can transfer freely
EvolutCoin	(No burn function)	–	–	N/A	Supply can never decrease. NatSpec claims a “deflationary-friendly” model but no burn mechanism exists on-chain. This is a permanent design constraint given contract immutability

Closing Summary

In this report, we have assessed the security of the EVOLUT Coin smart contract. The codebase appears to be well implemented.The audit was conducted in accordance with the procedure described above, including a thorough review of the contract logic and security considerations.

Based on our analysis, the code appears to be secure and no issues were identified during the audit.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

QuillAudits

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

March 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com